Ringoban: a Sokoban clone in Applesoft

What it is

Ringoban is a clone of Sokoban, originally written in 1981 by Hiroyuki Imabayashi, and published by Thinking Rabbit in 1982. It has spawned many variations and copies over the years since.

Ringoban is programmed entirely in Applesoft BASIC. The object is to push the boxes (represented in the game by the open-apple symbol) in a series of mazes onto marked target spaces. The player controls an "animated" pusher (the original eponymous warehouse man) who can only push boxes, and only one at a time.

Simple in concept, this version sticks closely to the original. The different mazes are referred to as levels. Only one move may be undone, though positions can be saved mid-level so you don't have to completely re-start a level after a mistake.

Because the game is programmed in BASIC, it can feel a bit sluggish on a 1 MHz machine. I didn't appreciate that as I was using emulation and a stock iigs to write/test Ringoban. Fortunately the person who brought this to my attention suggested compiling. Thanks, Daniel :). I'd never used Beagle Compiler, but once I tried it, it was great. The version on the associated disk has been compiled using the Beagle Compiler, and feels quite responsive.

Technical Notes

Setup

The program begins by initialising arrays and variables. A$(22) is used for the levels in string format, and a numeric array LV(22,32) is also used to save the level as numeric values. Levels can have a maximum of 22 rows and 32 columns; A$(0) is unused. Two other important arrays are F$(), used to store levels names, and LE() to store levels End_Of_File values.

After an introduction, the levels directory (LV$) is opened and the level names read into the array F$(), with the End_Of_File value stored in LE().

```
1997 :
1998 REM read directory
1999 :
2000 HOME : VTAB 10: HTAB 10: PRINT HG$" reading directory "HG$: PRINT
2010 PRINT D$"OPEN"LV$",TDIR": PRINT D$"READ"LV$
2020 INPUT N1$,N2$,N$: N1$ = N1$ + "/"
2030 INPUT N$: IF N$ = "" THEN PRINT: GOTO 2050
2040 N = N + 1:F$(N) = LEFT$ (N$,16): LE(N) = VAL ( MID$ (N$,69,3)): VTAB 14: PRINT ">
     ";: GOTO 2030
2050 PRINT D$"CLOSE"
```

Once the levels names are read in, the player has the choice of which level to play. Previously solved levels are marked with an '*' (ie, locked) but can be replayed. (The level selection routine is adapted from a menu program I found on a BRCC disk.)

After the player selects a level, it is loaded into memory...

```
1197 :
1198 REM read level
1199 :
```

```
1200 HOME
1210 A$(0) = LE$: I = 1: VTAB 10: PRINT "Reading level..."
1220 PRINT D$"open"LE$
1230 PRINT D$"read"LE$
1240 FOR J = 1 TO LE
1250 GET X$: B$ = B$ + X$
1260 IF ASC (X$) < > 13 THEN 1280
1270 A$(I) = B$: B$ = "": I = I + 1: VTAB 14: PRINT "> ";
1280 NEXT
1290 PRINT D$"close"
```

... centered horizontally and vertically (unnecessary, but vastly improves the appearance and hence the experience) ...

```
1359 :
1360 REM centre level
1361 :
1370 HH = 0: FOR J = 1 TO 22: LL = LEN (A$(J)) - 1: IF HH < LL THEN HH = LL
1380 NEXT : IF HH > 31 THEN 1400
1385 HH = INT ((32 - HH) / 2): IF HH < 1 THEN 1400
1390 IF I > 22 THEN I = 22
1395 FOR J = 1 TO I: A$(J) = LEFT$ (PA$,HH) + A$(J): NEXT
1400 IF I < 21 THEN VV = INT ((22 - I) / 2): FOR J = I TO 1 STEP - 1: A$(J + VV) =
     A$(J): NEXT
1410 IF VV > 0 THEN FOR J = 1 TO VV: A$(J) = "": NEXT
```

Note: PA$ (line 1395) is 16 spaces long; it is used to provide enough spaces to the start of each A$() for horizontal centering; 1400 and 1410 handle vertical centering.

... and then displayed on the screen ready to start:

```
1447 :
1448 REM display level
1449 :
1450 HOME : VTAB 23: PRINT "Displaying level..."
1455 VTAB 1: HTAB 1: BO = 0: BC = 0: MO = 0: PU = 0
1460 FOR Y = 1 TO 22: FOR X = 1 TO LEN (A$(Y)): B$ = MID$ (A$(Y),X,1)
1470 IF B$ = "#" THEN LV(Y,X) = 0: IF (X / 2 = INT (X / 2)) THEN PRINT W1$;: GOTO 1490
1480 IF B$ = "#" THEN PRINT W2$;
1490 IF B$ = " " THEN LV(Y,X) = 1: PRINT B$;
1500 IF B$ = "$" THEN LV(Y,X) = 2: PRINT BO$;
1510 IF B$ = "." THEN LV(Y,X) = 3: PRINT ".";$;:BO = BO + 1
1520 IF B$ = "*" THEN LV(Y,X) = 4: PRINT BC$;:BO = BO + 1:BC = BC + 1
1530 IF B$ = "@" THEN LV(Y,X) = 1: PRINT P2$;:VT = Y:HT = X
1540 IF B$ = "+" THEN LV(Y,X) = 3: PRINT ".";:BO = BO + 1:VT = Y:HT = X
1550 NEXT: PRINT: NEXT
```

Note lines 1470 and 1480 - there are two different mousetext characters used for walls, depending whether HTAB (the X value) is odd or even. This is done purely for cosmetic reasons.

Line 1500 - '$' is a box, so prints open-apple (Ꭿ)
Line 1510 - '.' is a target, so prints "." and increments number of boxes
Line 1520 - '*' is a box on target, (Ꭾ) and increments the number of boxes on target

Line 1530 - '@' is the player (but see 1540) and gets start VTAB, HTAB values
Line 1540 - '+' is the player on target space, so is treated as a target and sets the start VTAB, HTAB values of the player, since line 1530 would have been skipped.

The setup is concluded by printing the stats text on the right and command keys at the bottom of the screen:

```
1560 VTAB 2: HTAB 33: PRINT "Ring";BO$;"b";BC$;"n"
1565 VTAB 3: HTAB 33: PRINT M1$"LLLLLLL"M0$
1570 VTAB 5: HTAB 34: PRINT "Level": VTAB 6: HTAB 35: PRINT C
1580 VTAB 8: HTAB 34: PRINT "Moves"
1590 VTAB 11: HTAB 34: PRINT "Pushes"
1600 VTAB 14: HTAB 34: PRINT "Boxes"
1610 VTAB 17: HTAB 34: PRINT "Placed"
1620 VTAB 23: CALL - 958: PRINT "  ";M1$;"K";M0$;"   i   z: undo   n: new   s: store":
     HTAB 2: PRINT M1$;"HJU";M0$;: HTAB 6: PRINT "jkl  u: reset  q: quit  r: restore";
1630 RETURN
```

and play is ready to start. Walls, open apples, closed apples, targets and the player token all have VTAB and HTAB values.

This is Level 1, as the text file and as the numeric array (before centering):

```
A$(1)        #####              LV(1,...)      1111100000
A$(2)        #   #              LV(2,...)      1111101110
A$(3)        #$  #              LV(3,...)      1111102110
A$(4)      ###  $##             LV(4,...)      11100011200
A$(5)      #  $ $ #             LV(5,...)      11101121210
A$(6)   #### # ## #   ######    LV(6,...)      000010100101110000000
A$(7)   #    # ## #####  ..#    LV(7,...)      011110100100000113330
A$(8)   # $  $           ..#    LV(8,.. )      012112111111111113330
A$(9)   ##### ### ##@##  ..#    LV(9,...)      000001000100100113330
A$(10)      #     ##########    LV(10,...)     111101111100000000000
A$(11)      #######              LV(11,...)     11110000000
```

Note:
1 - that for lines 1 - 5, 10, and 11 there are leading spaces. The program treats these as floor spaces, but because of the placement of walls, they will never be used. Trailing spaces are not used.
2 - the player token in A$(9) does not have an LV() value. It needs only VTAB and HTAB values, which are determined in either line 1530 or 1540
3 - it would be possible to have all levels use the full 22 rows, 32 columns, and load them in already centered. Too much work formatting the levels!

Gameplay

Important note: in the situations described below, I have given the original listing as I wrote it. In many cases, lines could be combined to save memory and/or increase speed. I wrote for clarity in the first instance, so they came out as they did.

Once the main game loop begins, current stats are displayed - moves, pushes, total boxes, and boxes on target. The initial position of the player is VTAB VT, HTAB HT. Depending on whether the number of turns is odd or even, the player symbol changes, to give an animation effect.

```
97 :
98 REM display stats
99 :
100 VTAB VT: HTAB HT: IF (T / 2 = INT (T / 2)) THEN PRINT P2$;: GOTO 130
110 PRINT P1$;
130 VTAB 9: HTAB 35: PRINT MO;: VTAB 12: HTAB 35: PRINT PU;
140 VTAB 15: HTAB 35: PRINT BO;: VTAB 18: HTAB 35: PRINT BC:
147 :
148 REM win?
149 :
150 IF BC = BO THEN GOTO 800
```

Once play begins, there are two possibilities for each turn - move/push, or a command key.

```
197 :
198 REM get K$
199 :
200 VTAB 24: HTAB 1: IF PEEK (-16384) < 128 THEN 200
210 GET K$: K = ASC(K$) - 32 * (ASC(K$) > 96)
220 IF K = 11 OR K = 73 THEN V = -1: H = 0: GOTO 350: REM up
230 IF K = 10 OR K = 75 THEN V = 1: H = 0: GOTO 350: REM down
240 IF K = 8 OR K = 74 THEN V = 0: H = -1: GOTO 350: REM left
250 IF K = 21 OR K = 76 THEN V = 0: H = 1: GOTO 350: REM right
```

Lines 200 & 210 handle input. The cursor is suppressed, and input can be either lowercase or uppercase. If a movement key is pressed, then V and H are set accordingly and a check is made to see if that move is possible.

Move/Push logic

For both of these, the first step is to check if the input is valid. This is done in line 350.

```
347 :
348 REM process move
349 :
350 IF (LV(VT + V,HT + H) = 0) OR (LV(VT + V,HT + H) = 2 AND (LV(VT + V + V,HT + H + H)
     = 0 OR LV(VT + V + V,HT + H + H) = 2 OR LV(VT + V + V,HT + H + H) = 4)) OR (LV(VT +
     V,HT + H) = 4 AND (LV(VT + V + V,HT + H + H) = 0 OR LV(VT + V + V,HT + H + H) = 2
     OR LV(VT + V + V,HT + H + H) = 4)) THEN 200
```

This is, I think, the most complicated line in the whole program; certainly the longest! I had to add this line after the first release of the program. It checks that a push actually can be done. The program does work just fine without this line, BUT it is needed to prevent some very strange behaviour that can be triggered in the undo routine.

A move is really easy to process:

```
355 Z = 1: IF LV(VT + V,HT + H) = 1 THEN MO = MO + 1: GOTO 540
360 IF LV(VT + V,HT + H) = 3 THEN MO = MO + 1: GOTO 540
```

Set Z flag to 1 (needed for the undo function), then check if moving to a floor space (line 355) or target (line 360). Either way, the number of moves is incremented and the program branches to 540. Lines 355 and 360 are both conditional; if the conditions are not met, then it is a push and falls through to line 380. The final tidy-up is in lines 540 - 560; since they are common to both move and push they are explained below.

A push is more complicated:

```
369 :
370 REM process push
371 :
380 Z = 2
390 IF LV(VT + V,HT + H) = 2 AND LV(VT + V + V,HT + H + H) = 1 THEN 440
400 IF LV(VT + V,HT + H) = 2 AND LV(VT + V + V,HT + H + H) = 3 THEN 460
410 IF LV(VT + V,HT + H) = 4 AND LV(VT + V + V,HT + H + H) = 1 THEN 480
420 IF LV(VT + V,HT + H) = 4 AND LV(VT + V + V,HT + H + H) = 3 THEN 500
430 GOTO 200
440 LV(VT + V + V,HT + H + H) = 2
450 VTAB VT + V + V: HTAB HT + H + H: PRINT BO$;: GOTO 520
460 LV(VT + V + V,HT + H + H) = 4
```

```
470 VTAB VT + V + V: HTAB HT + H + H:BC = BC + 1: PRINT BC$;: GOTO 520
480 LV(VT + V + V,HT + H + H) = 2
490 VTAB VT + V + V: HTAB HT + H + H:BC = BC - 1: PRINT BO$;: GOTO 520
500 LV(VT + V + V,HT + H + H) = 4
510 VTAB VT + V + V: HTAB HT + H + H: PRINT BC$;
520 LV(VT + V,HT + H) = 1:PU = PU + 1
530 IF MID$ (A$(VT + V),HT + H,1) = "." OR MID$ (A$(VT + V),HT + H,1) = "*" OR MID$
     (A$(VT + V),HT + H,1) = "+" THEN LV(VT + V,HT + H) = 3
540 IF MID$ (A$(VT),HT,1) = "." OR MID$ (A$(VT),HT,1) = "*" OR MID$ (A$(VT),HT,1) = "+"
     THEN VTAB VT: HTAB HT: PRINT ".";: GOTO 560
550 VTAB VT: HTAB HT: PRINT " ";
560 VT = VT + V: HT = HT + H: T = T + 1: VM = V: HM = H: GOTO 100
```

Line 380 sets the undo flag to 2 so the undo routine can process it correctly, then:

In a valid push, there are four possibilities, lines 390 - 420:

1. box pushed onto a floor space (lines 390, 440, and 450)
   - line 440 - yes, so update numeric array for the box
   - line 450 - print appropriate box symbol then GOTO 520
2. box pushed onto a target (lines 400, 460, and 470)
3. box pushed off a target onto a floor space (lines 410, 480, and 490)
4. box pushed off a target onto a target (lines 420, 500, and 510)

(Line 430 loops back to get a new keypress; actually it became superfluous when I added line 350.)

Line 520 updates the numeric array - although line 530 can re-assign it - and increases the push count by 1.

Final tidy-up - line 540 checks the vacated space by referencing the A$() array. If it is a target space, print '.' else print a space. Then in line 560, set new values for the player character, increment turn number by 1 (used to control the animation), store V and H in VM and HM respectively (needed for the undo function), and go get the next keypress.

Undo Move/Push Subroutine

Undoing a move is quite straightforward. The V and H values from the move just made are still in memory, so the only thing is to check whether the space moved off of is a floor or target - there are no other alternatives.

```
597 :
598 REM undo
599 :
600 V = VM: H = HM: IF Z = 0 THEN 200
610 IF Z = 2 THEN 670
619 :
620 REM undo move
621 :
630 LV(VT,HT) = 1: VTAB VT: HTAB HT: PRINT " "
640 IF MID$ (A$(VT),HT,1) = "." OR MID$ (A$(VT),HT,1) = "+" OR MID$ (A$(VT),HT,1) = "*"
     THEN LV(VT,HT) = 3: VTAB VT: HTAB HT: PRINT "."
650 MO = MO - 1: GOTO 760
```

Line 600 - V and H are got from VM and HM, and if undo not permitted, go get a valid keypress.
Line 610 if Z = 2 then undoing a push, goto 670.
Line 630 sets correct VTAB and HTAB values for the player, and makes the vacated space a floor. Line 640 covers the case that the vacated space is a target. Note that a target space in A$() can be either "."

(target), "+" (player starting on a target), or "*" (box on a target). Then 650 decreases the move count and branches to 760 to set values for VTAB and HTAB, decrement (is that a real word?) T and set undo flag Z to 0.

Undoing a push is more complicated as there are more spaces to consider.

```
659 :
660 REM undo push
661 :
670 IF LV(VT + V,HT + H) = 4 THEN LV(VT + V,HT + H) = 3
680 IF LV(VT + V,HT + H) = 2 THEN LV(VT + V,HT + H) = 1: GOTO 720
690 VTAB VT + V: HTAB HT + H: PRINT ".": VTAB VT: HTAB HT: PRINT BO$
700 BC = BC - 1: LV(VT,HT) = 2
710 GOTO 740
720 VTAB VT + V: HTAB HT + H: PRINT " ": VTAB VT: HTAB HT: PRINT BO$
730 LV(VT,HT) = 2
740 IF MID$ (A$(VT),HT,1) = "." OR MID$ (A$(VT),HT,1) = "*" OR MID$ (A$(VT),HT,1) = "+"
     THEN VTAB VT: HTAB HT: PRINT BC$: LV(VT,HT) = 4: BC = BC + 1
750 PU = PU - 1
760 VT = VT - V: HT = HT - H: T = T - 1: Z = 0: RETURN
```

Starting in line 670 - if the box being pulled back on is on a target, give it value 3 in LV(), or (line 680) if not on a target, set value as 1, goto 720.
Line 690 is reached if the condition in line 670 is met, so print a target space, print a box in the space it came from, and (line 700) reduce BC (boxes on targets) by 1, set correct LV() values, and then (line 710) goto line 740 for final cleanup.
Lines 720 and 730 do almost the same, except a space is printed, BC is not reduced, and LV() is made 1 - a floor space.
Line 740 checks what the space the box being restored is going onto - if a target (".", "+", or "*") then print the appropriate symbol (BC$), and increase the BC count. Finally, reduce the push count (line 750).

Line 760 is common to undo both move and push; it concludes the undo routine - reset VT, HT, T and Z, and RETURN.

Store/Restore

ProDOS has a very handy command in STORE. It takes a snapshot of all current variables and their values and saves it all to disk. RESTORE reads them all back into memory. I use these commands to save a position. The STORE/RESTORE commands could also be used as a save game feature. Maybe in a future version?

```
1697 :
1698 REM store
1699 :
1700 VTAB 23: CALL - 958
1705 PRINT "Saving position...": ST = 1
1710 PRINT D$"store ringo"
1715 GOSUB 1620
1720 RETURN
```

A very simple bit of code. Clear the text on the last two lines of the screen, set the save game flag to 1, restore the text on the bottom two lines (GOSUB 1620) and it's done.

Restore is slightly more complicated. First, check there is a position to restore; if not, put the text back on screen and RETURN.

```
1797 :
1798 REM restore
```

```
1799 :
1800 VTAB 23: CALL  - 958: IF ST THEN 1830
1805 PRINT " No position saved.";: FOR I = 1 TO 1000: NEXT : PRINT " Press return."
1810 IF PEEK (-16384) < 128 THEN 1810
1815 GET K$: K = ASC(K$) - 32 * (ASC(K$) > 96)
1820 IF K = 13 THEN 1870
1825 GOTO 1800
1830 PRINT "Restoring..."
1835 PRINT D$"restore ringo"
1840 FOR I = 1 TO 22: FOR J = 1 TO 32
1845 IF LV(I,J) = 1 THEN  VTAB I: HTAB J: PRINT " "
1850 IF LV(I,J) = 2 THEN  VTAB I: HTAB J: PRINT BO$
1855 IF LV(I,J) = 3 THEN  VTAB I: HTAB J: PRINT "."
1860 IF LV(I,J) = 4 THEN  VTAB I: HTAB J: PRINT BC$
1865 NEXT : NEXT
1870 GOSUB 1620
1875 RETURN
```

Once the variables have been read back into memory, it is necessary to re-draw the screen using those values. This is done in lines 1840 - 1865. There is no need to redraw walls, as they are fixed. Restore the text at the bottom of the screen, and continue play from the restored position. There is scope here for reducing the number of iterations, depending on the size of the level (rows and columns).

Final Notes.

Thank you for being interested enough to read this far! When I wrote Ringoban I did so for my own enjoyment - a good project, and I wanted to play it. I am a bit surprised and very happy that others are interested in it. Because of that, it has grown beyond what I first had, and hopefully will keep growing
A special thanks to Daniel Henderson for his promoting the game, and Bill Martens for putting it on Call A.P.P.L.E., and also to the members of CALL -151, BASIC Programming Language, and Sokoban for encouragement and comments.